



UNIVERSITÉ HASSAN II DE CASABLANCA
École Normale Supérieure de l'Enseignement Technique
Mohammedia

HARDEN v4.0

Linux Audit, Hardening & Remediation Suite

COMPTE RENDU — MINI PROJET SHELL

Module	Théorie des Systèmes d'Exploitation & Linux
Encadrant	ENSET Mohammedia
TeamID	_____
Membres	_____

Date	14/05/2026
Version script	4.0 (14-05-2026)

Année Universitaire 2025-2026

Table des matières

1	Introduction et besoin identifié	3
1.1	Contexte général	3
1.2	Besoin identifié	3
1.3	Objectifs du script	3
1.4	Public cible	3
1.5	Architecture générale	4
2	Fonctionnalités implémentées	5
2.1	Syntaxe d'utilisation	5
2.2	Options disponibles	5
2.3	Vérifications de sécurité — <code>full_scan</code> (20 checks)	6
2.4	Gestion des logs	7
2.5	Helpers C compilés à la volée	8
2.5.1	<code>fork_helper.c</code> — mode <code>-f</code>	8
2.5.2	<code>thread_helper.c</code> — mode <code>-t</code>	8
2.6	Concepts shell implémentés	9
2.7	Sécurité interne du script	9
3	Tests et exécutions	11
3.1	Mode normal — Scan complet (<code>-C</code>)	11
3.2	Mode subshell (<code>-s</code>)	11
3.3	Mode fork (<code>-f</code>)	12
3.4	Mode thread (<code>-t</code>)	12
3.5	Tableau comparatif des performances	13
3.6	Fonctionnalités avancées	13
3.6.1	Baseline de fichiers critiques	13
3.6.2	Auto-remédiation	13
3.6.3	Analyse CVE kernel	14
3.6.4	Export JSON et Banner Grab	14
4	Gestion des erreurs	15
4.1	Tableau des codes d'erreur	15
4.2	Code 100 — Option invalide	15
4.3	Code 101 — Paramètre manquant	15
4.4	Code 102 — Permission denied	16
4.5	Code 103 — Erreur de log	16
4.6	Codes 104 et 105 — Python / Compilation C	16
5	Documentation	17
5.1	Documentation intégrée (<code>-h</code>)	17
5.2	Structure de la documentation intégrée	17
5.3	Bannière ASCII art	17
5.4	Résumé de fin de scan	18
6	Conclusion	19

1 Introduction et besoin identifié

1.1 Contexte général

Dans le domaine de l'administration système Linux, la sécurité des serveurs et postes de travail est une préoccupation permanente. Les attaques par élévation de privilèges, les mauvaises configurations et les binaires vulnérables constituent des vecteurs d'attaque fréquemment exploités par des acteurs malveillants.

La détection manuelle de ces risques est longue, sujette aux erreurs humaines et difficilement reproductible à grande échelle. L'automatisation de ces tâches d'audit représente donc un enjeu majeur pour toute organisation soucieuse de sa posture de sécurité.

1.2 Besoin identifié

HARDEN est un outil d'audit et de durcissement (*hardening*) de sécurité Linux développé en Bash. Il automatise la détection des risques d'élévation de privilèges, des mauvaises configurations et des vulnérabilités courantes sur les systèmes Unix/Linux.

L'outil répond à un besoin réel des administrateurs système, des équipes de sécurité (Blue Team / Red Team) et des ingénieurs DevSecOps qui doivent effectuer des audits réguliers sur un ou plusieurs systèmes Linux.

1.3 Objectifs du script

- Détecter automatiquement les binaires SUID vulnérables référencés dans GTFOBins
- Identifier les règles sudo dangereuses (NOPASSWD, wildcards)
- Analyser les tâches cron, les fichiers mal configurés et les clés SSH exposées
- Vérifier la configuration SSH, les capacités kernel et les permissions Docker
- Générer des logs horodatés avec des recommandations de correction (**Fix:**)
- Supporter plusieurs modes d'exécution : normal, subshell, fork réel (C/`fork()`) et thread réel (C/pthreads)
- Fournir des fonctionnalités avancées : baseline SHA-256, auto-remédiation, export JSON, banner grabbing, analyse CVE kernel

1.4 Public cible

L'outil s'adresse principalement aux :

- Administrateurs système Linux et ingénieurs DevSecOps
- Équipes de sécurité (Blue Team pour l'audit défensif, Red Team pour l'identification de failles)
- Auditeurs de sécurité souhaitant automatiser les contrôles de conformité
- Étudiants souhaitant comprendre et renforcer la posture de sécurité d'un système Linux

1.5 Architecture générale

HARDEN est structuré autour d'un script principal Bash (`harden.sh`) et de deux helpers C compilés dynamiquement dans un répertoire sécurisé :

- `fork_helper.c` — Implémente le vrai appel système `fork()` pour le mode `-f` (background réel).
- `thread_helper.c` — Utilise les POSIX threads (`pthread_create/join`) pour le mode `-t`.

L'architecture repose sur des mesures de sécurité internes robustes :

- **Gestion des logs** : Utilisation de `flock` sur `history.log` pour éviter les race conditions en mode concurrent.
- **Isolation du Build** : Utilisation de `mktemp -d` pour `BUILD_DIR` afin de prévenir les attaques par lien symbolique.
- **Nettoyage automatique** : Un `trap '...' EXIT` garantit la suppression des fichiers temporaires à la fin du script.
- **Codes d'erreur standardisés** : Gestion des erreurs via des codes spécifiques (100–105).

L'architecture des fonctions de scan est modulaire :

```
./harden [options] <repertoire>
-c --> quick_scan()      # 7 checks essentiels
-C --> full_scan()       # 20 checks complets
-s --> ( full_scan )     # sous-shell isole
-t --> thread_helper "cmd1" "cmd2" # Thread1=checks 1-11 /
    Thread2=12-20
-f --> fork_helper "$0 _real_fork_target" & # fork reel
```

Cette architecture en deux couches (Bash + C) respecte la directive du cahier des charges sur l'utilisation de « *scripts complémentaires en langage C* ».

2 Fonctionnalités implémentées

2.1 Syntaxe d'utilisation

Le script HARDEN respecte la syntaxe standard des commandes Linux (directive §3.2.4) :

```
./harden.sh [OPTIONS] <répertoire>
```

Le paramètre <répertoire> est **obligatoire** pour les modes de scan (**-c**, **-C**, **-t**, **-s**, **-f**). Il représente le répertoire cible sur lequel les vérifications sont effectuées.

2.2 Options disponibles

Le script implémente les options suivantes, conformément aux directives §3.2.2 du cahier des charges :

Option	Nom	Description
-h	Help	Affiche la documentation complète (usage, options, codes d'erreur, exemples)
-c	Quick scan	Scan rapide (7 vérifications essentielles) sur le répertoire cible
-C	Full scan	Scan complet (20 vérifications) via <code>full_scan()</code> sur le répertoire cible
-t	Thread	Exécution parallèle via de vrais POSIX threads (C helper compilé avec pthread)
-s	Subshell	Exécute le scan dans un sous-shell isolé via (<code>full_scan</code>)
-f	Fork	Exécution concurrente (Père scanne Dir1, Fils scanne Dir2) via <code>fork()</code> réel (C helper). Le fils tourne en arrière-plan (<code>disown</code>) et silencieusement.
-l DIR	Log directory	Spécifie un répertoire personnalisé pour le fichier <code>history.log</code>
-r	Restore	Réinitialise les paramètres par défaut — nécessite root (<code>\$EUID=0</code>)
-n	Dry-run	Simulation : aucun log écrit sur disque (compteurs actifs en mémoire)
-v	Verbose	Active le mode verbeux pour afficher des détails supplémentaires
-V	Version	Affiche la version du programme et quitte
-a	Auto-remediation	Extrait du log et applique automatiquement les corrections (Fix:)
-b store	Baseline store	Calcule et enregistre les hash SHA-256 des fichiers critiques

Option	Nom	Description
-b check	Baseline check	Compare l'état actuel aux hash stockés pour détecter les modifications
-B	Banner grab	Récupère les bannières des ports 22, 80, 443 via socket Python
-k	Kernel CVE	Analyse hors-ligne le noyau pour identifier les CVE connus
-j	JSON export	Exporte history.log en JSON structuré via Python

*Note : Les options **-t**, **-s**, **-f** et **-c/-C** requièrent un <répertoire> cible. Les options **-a**, **-b**, **-B**, **-k**, **-j** sont des actions autonomes sans répertoire requis.*

2.3 Vérifications de sécurité — full_scan (20 checks)

Le scan complet (**-C**) exécute **20 vérifications** via **full_scan()**. Le scan rapide (**-c**) via **quick_scan()** exécute 7. En mode **-t**, les checks 1–11 s'exécutent dans le Thread 1, les checks 12–20 dans le Thread 2 :

#	Fonction	Description
1	check_suid	Binaires SUID inconnus dans \$TARGET (Audit localisé)
2	check_suid_vulnerable	SUID vulnérables (GTFOBins) — Scan find / système-complet
3	check_sudo	Permissions sudo via sudo -n -l (Système-complet)
4	check_sudo_nopasswd	Règles NOPASSWD dans /etc/sudoers* (Système-complet)
5	check_cron	Tâches cron de l'utilisateur courant (Système-complet)
6	check_cron_paths	PATH non sécurisés dans les tâches cron (Système-complet)
7	check_passwd	Permissions de /etc/passwd (doit être 644) (Système-complet)
8	check_root_users	Comptes non-root avec UID 0 (Système-complet)
9	check_empty_passwd	Hash vide dans /etc/shadow (Système-complet)
10	check_env	Variables dangereuses (\$PATH='.' , \$LD_PRELOAD) (Système-complet)
11	check_ssh	Clés SSH privées exposées dans \$TARGET (Audit localisé)
12	check_ssh_config	sshd_config (PermitRootLogin, etc.) (Système-complet)

#	Fonction	Description
13	<code>check_permissions</code>	Fichiers world-writable dans <code>\$TARGET</code> (Audit localisé)
14	<code>check_world_writable</code>	<code>/tmp</code> , <code>/var/tmp</code> , <code>/dev/shm</code> (Système-complet)
15	<code>check_unowned_files</code>	Fichiers sans propriétaire (Système-complet)
16	<code>check_writable_binaries</code>	Binaires modifiables dans <code>/bin</code> <code>/sbin</code> <code>/usr/bin</code> (Système-complet)
17	<code>check_tmp_mounts</code>	Options de montage <code>noexec</code> , <code>nosuid</code> sur <code>/tmp</code> (Système-complet)
18	<code>check_kernel_version</code>	Version du noyau courant (Système-complet)
19	<code>check_capabilities</code>	Capacités Linux sur les binaires (<code>getcap -r /</code>) (Système-complet)
20	<code>check_docker_socket</code>	Permissions de <code>/var/run/docker.sock</code> (Système-complet)

Actions autonomes (hors `full_scan`) : `-a` auto-remédiation, `-b store/check` baseline SHA-256 — invoquées indépendamment, pas lors d'un scan.

2.4 Gestion des logs

Toutes les sorties sont simultanément affichées dans le terminal (codes couleur ANSI) et enregistrées dans `history.log` après stripping des couleurs via `sed`. Le format respecte strictement la directive §3.2.2 :

```
yyyy-mm-dd-hh-mm-ss : username : INFOS/WARNING/CRITICAL :
    message
```

Améliorations techniques du logging :

- **Concurrence sécurisée** : Utilisation de `flock` sur le fichier log pour garantir l'intégrité des données lors d'exécutions parallèles (Thread/Fork).
- **Traçabilité Fork** : Les messages du processus fils sont tagués `[FORK:PID=...]` pour une identification claire.
- **Compteurs persistants** : Les compteurs d'incidents sont toujours incrémentés, même en mode dry-run (`-n`).

L'emplacement du fichier log s'adapte automatiquement selon les privilèges :

- **root** : `/var/log/harden/history.log`
- **non-root** : `~/.harden/history.log`
- **option `-l DIR`** : répertoire personnalisé spécifié par l'utilisateur

Contenu du fichier *history.log* après un scan complet — niveaux *INFOS* / *WARNING* / *CRITICAL*

Affichage coloré dans le terminal — codes couleur ANSI verts, jaunes, rouges

2.5 Helpers C compilés à la volée

Conformément à la directive §3.2.2 permettant des « *scripts complémentaires en langage C* », HARDEN utilise deux helpers C compilés automatiquement avec `gcc` :

2.5.1 `fork_helper.c` — mode `-f`

Ce helper implémente un vrai appel système `fork()` (via la fonction POSIX `fork(2)`). Le processus parent lance le fils et continue son exécution, tandis que le fils s'exécute en arrière-plan (`disown`) avec une redirection silencieuse de ses sorties standards.

```
pid_t pid = fork();
if (pid == 0) {
    execl("/bin/sh", "sh", "-c", argv[1], NULL);
} else {
    int status;
    waitpid(pid, &status, 0);
}
```

2.5.2 `thread_helper.c` — mode `-t`

Ce helper crée de vrais POSIX threads via `pthread_create()` avec `libpthread`. Les deux threads reçoivent des **commandes distinctes** (`argv[1]` et `argv[2]`) pour un vrai découpage fonctionnel, puis `pthread_join()` synchronise leur fin.

```
// thread_helper.c : 2 commandes distinctes
typedef struct { char *command; } thread_data_t;
int main(int argc, char *argv[]) {
    pthread_t threads[2];
    thread_data_t data[2];
    data[0].command = argv[1]; /* Thread 1 : checks 1-11 */
    data[1].command = argv[2]; /* Thread 2 : checks 12-20 */
    pthread_create(&threads[0], NULL, run_command, &data[0]);
    pthread_create(&threads[1], NULL, run_command, &data[1]);
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);
    return 0;
}
```


2.6 Concepts shell implémentés

Conformément à la directive §3.2.2, le script incorpore les concepts suivants :

Concept shell	Implémentation dans HARDEN
Conditions (<code>if/else</code> , <code>[]</code>)	Vérification <code>\$EUID</code> , tests <code>-f/-d/-w</code> , comparaisons de permissions
Boucles (<code>while read</code> , <code>for</code>)	<code>while read -r file (find/grep)</code> , <code>for vuln in "\${vulnerable[@]}"</code>
Fonctions modulaires	30+ fonctions : <code>check_*</code> , <code>log_*</code> , <code>quick_scan</code> , <code>full_scan</code> , <code>thread_mode</code> , etc.
Variables d'environnement	Contrôle de <code>\$PATH</code> (contient <code>'</code>), <code>\$LD_PRELOAD</code> , <code>\$HOME</code> , <code>\$EUID</code>
Expressions régulières	<code>grep -E, =~</code> pour la version kernel, patterns Bash dans <code>getopts</code>
Manipulation de fichiers	<code>find</code> , <code>stat -c %a</code> , <code>chmod</code> , <code>sha256sum</code> , <code>mkdir -p</code>
Contrôle d'accès	<code>require_root()</code> avec <code>EUID=0</code> , whitelist binaires SUID protégés
Pipes et filtres	<code>find while read</code> , <code>grep awk '{print \$1}'</code> , <code>sed</code> , <code>cut</code> , <code>sort -u</code>
Sous-shells (<code>()</code>)	<code>(full_scan)</code> pour le mode <code>-s</code> , isolation de l'environnement
Processus fils (<code>fork</code>)	<code>fork_helper.c</code> compilé par <code>gcc</code> dans <code>/tmp</code> , simple <code>&</code>
Threads POSIX	<code>thread_helper.c</code> compilé avec <code>-lpthread</code> , 2 threads avec tâches distinctes
Scripts C complémentaires	<code>fork_helper.c</code> et <code>thread_helper.c</code> générés et compilés automatiquement

2.7 Sécurité interne du script

Le script HARDEN v4.0 intègre des mécanismes d'auto-protection pour garantir son intégrité et sa robustesse :

- **Protection contre les liens symboliques** : Le répertoire de compilation `BUILD_DIR` est créé via `mktemp -d`, garantissant un nom imprévisible et des permissions restreintes dans `/tmp`.
- **Sûreté de l'auto-remédiation** : L'utilisation de `eval` a été proscrite. Les commandes sont analysées et filtrées via une whitelist stricte (`chmod/chown`).
- **Protection des binaires système** : Une liste d'exclusion (`PROTECTED_BINS`) empêche toute modification accidentelle ou malveillante des binaires de sécurité vitaux (`sudo`, `passwd`, etc.).

- **Exécution non-interactive** : L'utilisation de `sudo -n` dans les tests de privilèges évite tout blocage du script en attente d'une saisie utilisateur.
- **Verrouillage atomique** : Le recours à `flock` pour les écritures dans le journal prévient toute corruption du fichier `history.log` lors d'accès concurrents.

3 Tests et exécutions

Conformément à la directive §3.2.4, les quatre modes d'exécution ont été testés avec la commande `time` sur trois scénarios : léger (`/home`), moyen (`/usr`) et lourd (`/`). Ces tests permettent d'évaluer l'efficacité des différents modes de création de processus.

3.1 Mode normal — Scan complet (`-C`)

Le mode normal (`-C`) exécute séquentiellement les **20 vérifications** de `full_scan()` dans le processus courant. C'est le mode de référence pour comparer les performances.

Commandes utilisées

```
time ./harden -C /home      # Scénario léger
time ./harden -C /usr       # Scénario moyen
time ./harden -C /          # Scénario lourd
```

Exécution mode `-C` — scan léger sur `/home` — affichage terminal + résultat `time`

Exécution mode `-C` — scan lourd sur `/` — résultat `time` (référence de performance)

3.2 Mode subshell (`-s`)

Le mode subshell exécute `full_scan()` dans un sous-shell isolé via la syntaxe Bash (`full_scan`). Les variables modifiées dans le sous-shell n'affectent pas l'environnement parent. Ce mode illustre le concept d'isolation par sous-shell vu en cours.

Commandes utilisées

```
time ./harden -s /home
time ./harden -s /usr
time ./harden -s /
```

Exécution mode `-s` (subshell) — scan sur `/home` — comparaison avec mode `-C`

3.3 Mode fork (-f)

Le mode fork utilise le vrai appel système `fork()` via le helper C compilé dans `/tmp`. Le processus fils exécute `harden -C TARGET` via `execl()` et le PID est affiché dans les logs.

Mécanisme

```
# Bash (harden.sh) :
"${BUILD_DIR}/fork_helper" "$0 _real_fork_target $TARGET" &

# fork_helper.c :
pid_t pid = fork();
if (pid == 0) { execl("/bin/sh", "sh", "-c", argv[1], NULL); }
else { waitpid(pid, &status, 0); }
```

Commandes utilisées

```
time ./harden -f /home
time ./harden -f /usr
time ./harden -f /
```

Exécution mode -f (fork réel) — PID du processus fils visible dans les logs

3.4 Mode thread (-t)

Le mode thread utilise de vrais POSIX threads via le helper C (`thread_helper`) avec `libpthread`. Les deux threads reçoivent des **commandes distinctes** : Thread 1 exécute les checks 1–11, Thread 2 exécute les checks 12–20.

Mécanisme

```
// thread_helper.c :
data[0].command = argv[1]; /* "harden _thread_part1 $TARGET" */
data[1].command = argv[2]; /* "harden _thread_part2 $TARGET" */
pthread_create(&threads[0], NULL, run_command, &data[0]);
pthread_create(&threads[1], NULL, run_command, &data[1]);
pthread_join(threads[0], NULL);
pthread_join(threads[1], NULL);
```

Commandes utilisées

```
time ./harden -t /home
time ./harden -t /usr
time ./harden -t /
```

Exécution mode **-t** (POSIX threads réels) — gain de performance sur scan lourd

3.5 Tableau comparatif des performances

Temps d'exécution (commande **time**, valeur **real**) mesurés sur les trois scénarios :

Scénario	Normal -C	Subshell -s	Fork -f	Thread -t
Léger (/home)	_____ s	_____ s	_____ s	_____ s
Moyen (/usr)	_____ s	_____ s	_____ s	_____ s
Lourd (/)	_____ s	_____ s	_____ s	_____ s

À compléter : remplacer les blancs par les valeurs réelles mesurées lors des tests avec **time** sur votre système.

3.6 Fonctionnalités avancées

3.6.1 Baseline de fichiers critiques

```
./harden -b store      # Stocke les hash SHA-256 de /etc/passwd,
                       /etc/shadow,
                       # /etc/sudoers, /etc/crontab, /etc/ssh/
                       #      sshd_config
                       # et tous les binaires SUID trouvés
./harden -b check     # Détecte les modifications depuis la
                       baseline
```

Démonstration baseline **store** puis **check** — détection d'un fichier modifié

3.6.2 Auto-remédiation

```
sudo ./harden -a      # Extrait les lignes "Fix:" du log et les
                       applique
                       # après confirmation interactive (y/N)
```

L'implémentation de l'auto-remédiation dans la version 4.0 a été sécurisée :

- **Suppression de eval** : Prévention des injections de commandes via des logs malveillants.
- **Whitelist stricte** : Seules les commandes **chmod** et **chown** sont autorisées.
- **Protection système** : Les binaires critiques (/usr/bin/sudo, /usr/bin/passwd, etc.)

sont protégés contre toute modification automatique.

- **Backup systématique** : Création d'une sauvegarde dans `history.log/backup_TIMESTAMP` avant modification.

*Auto-remédiation : liste des commandes **Fix**: extraites et confirmation d'application*

3.6.3 Analyse CVE kernel

```
./harden -k      # Identifie Dirty Cow (CVE-2016-5195),  
                 # Dirty Pipe (CVE-2022-0847), PwnKit (CVE  
                 # -2021-4034)  
                 # selon la version de uname -r
```

Résultat de ./harden -k — CVE détectés selon la version du noyau

3.6.4 Export JSON et Banner Grab

```
./harden -j      # Exporte history.log en JSON structuré via  
                 Python  
./harden -B      # Banner grabbing sur les ports 22, 80 et 443  
                 via socket Python
```

Export JSON du log et résultat banner grab sur les ports locaux

4 Gestion des erreurs

Conformément à la directive §3.2.3, le programme gère activement les erreurs d'utilisation incorrecte. Chaque erreur déclenche :

1. Affichage coloré (rouge) sur `stderr`
2. Enregistrement dans le fichier `history.log`
3. Code de retour spécifique
4. Affichage automatique de la documentation (`-h`)

4.1 Tableau des codes d'erreur

Code	Nom	Description et déclenchement
100	Option invalide	Option non reconnue par <code>getopts</code> → <code>error_exit "Invalid option" 100</code>
101	Paramètre manquant	Répertoire cible absent ou invalide → <code>error_exit "Target directory required" 101</code>
102	Permission denied	Option <code>-r</code> sans <code>root</code> → <code>require_root()</code> → <code>error_exit 102</code>
103	Erreur de log	Impossible de créer le répertoire de log → <code>error_exit "Cannot create log dir" 103</code>
104	Python introuvable	Options <code>-B/-k/-j</code> sans <code>python3</code> → <code>error_exit "Python3 not found" 104</code>
105	Compilation échouée	<code>gcc</code> absent ou erreur de compilation → <code>error_exit "Failed to compile" 105</code>

4.2 Code 100 — Option invalide

Déclenché lorsqu'une option non reconnue est passée au script. `getopts` retourne '?' et la fonction `error_exit()` est appelée avec le code 100.

```
$ ./harden -z /home
```

Erreur code 100 : option invalide -z — message rouge sur stderr + aide automatique

4.3 Code 101 — Paramètre manquant

Déclenché dans deux cas : aucun répertoire fourni après les options, ou le chemin fourni n'est pas un répertoire valide.

```
$ ./harden -C # répertoire manquant  
$ ./harden -C /chemin/inexistant # répertoire invalide
```

Erreur code 101 : répertoire cible manquant ou invalide

4.4 Code 102 — Permission denied

Déclenché par l'option **-r** (restore) sans privilèges root. La fonction `require_root()` vérifie `$EUID` avant toute action.

```
$ ./harden -r # execute sans sudo (sans root)
```

Erreur code 102 : tentative de restore sans privilèges root

4.5 Code 103 — Erreur de log

Déclenché lorsque le script ne peut pas créer le répertoire de log spécifié via **-l** (permissions insuffisantes).

```
$ ./harden -l /root/logs -C /home # sans sudo
```

Erreur code 103 : impossible de créer le répertoire de log

4.6 Codes 104 et 105 — Python / Compilation C

Le code 104 est déclenché si `python3` est absent lors de l'utilisation des options **-B**, **-k** ou **-j**. Le code 105 est déclenché si `gcc` est absent ou si la compilation du helper C échoue.

```
$ ./harden -B # si python3 absent -> code 104  
$ ./harden -t /home # si gcc absent -> code 105
```

Erreurs codes 104 et 105 — Python introuvable ou compilation C échouée

5 Documentation

5.1 Documentation intégrée (-h)

Conformément à la directive §3.2.5, une documentation complète est intégrée dans le script et accessible via l'option **-h**. Elle couvre la syntaxe, toutes les options, les paramètres obligatoires, les codes d'erreur et des exemples concrets.

```
$ ./harden -h
```

Affichage complet de l'aide avec ./harden -h

5.2 Structure de la documentation intégrée

Élément	Contenu
En-tête	Nom HARDEN, version 4.0, date (14-05-2026)
Syntaxe	<code>harden [OPTIONS] [<directory>]</code>
Options	Liste des 16 options avec description courte (-h -c -C -t -s -f -l -r -n -v -V -a -b -B -k -j)
Paramètre	Description du répertoire cible (obligatoire pour -c/-C/-t/-s/-f)
Codes d'erreur	Tableau des codes 100–105 avec signification
Exemples	12 exemples d'utilisation couvrant tous les modes

5.3 Bannière ASCII art

HARDEN affiche une bannière colorée au démarrage de chaque scan, identifiant visuellement l'outil avec sa version (codes ANSI bleu) :

```
=====
HARDEN - Linux Audit, Hardening & Remediation Suite
Version: 4.0 (14-05-2026)
=====
```

Bannière HARDEN affichée au démarrage du scan avec couleurs ANSI

5.4 Résumé de fin de scan

À la fin de chaque scan, HARDEN affiche un résumé récapitulatif indiquant le nombre d'issues critiques, d'avertissements et de messages d'information, ainsi que les 3 derniers problèmes critiques détectés.

```
=== SUMMARY ===
Critical issues: 3
Warnings: 12
Info messages: 45
Log saved to: /var/log/harden/history.log

Top critical issues:
- Passwordless sudo rule: user ALL=(ALL) NOPASSWD:ALL
- Vulnerable SUID binary: /usr/bin/vim.basic
- /etc/passwd is writable
```

Résumé final affiché après un scan complet avec les issues critiques détectées

6 Conclusion

HARDEN v4.0 est un outil d'audit et de durcissement de sécurité Linux complet, développé en Bash avec des extensions en langage C. Il répond à un besoin réel des administrateurs système en automatisant la détection de **20 catégories de risques**, tout en respectant l'ensemble des directives du mini-projet.

Le script implémente avec succès les quatre modes d'exécution requis (normal, subshell, fork réel et thread réel POSIX), une gestion robuste des logs horodatés avec verrouillage via **flock**, ainsi qu'une gestion complète des erreurs avec six codes de retour spécifiques.

L'utilisation de helpers C compilés (**fork_helper.c** et **thread_helper.c**) représente une plus-value notable : les appels système **fork()** et **pthread_create()** sont réellement invoqués, garantissant une concurrence réelle et efficace.

Historique des modifications

- **v1.0 - v2.0** : Implémentation des checks de base et du logging séquentiel.
- **v3.0** : Introduction des helpers C pour fork et threads.
- **v4.0 (Actuelle)** : Intégration de **flock** pour le logging concurrent, sécurisation du **BUILD_DIR** avec **mktemp**, implémentation du mode fork avec **disown** et silence fils, et passage à 20 checks de sécurité complets.

- **20 vérifications** dans **full_scan** + **7** dans **quick_scan** — architecture modulaire **quick_scan** / **full_scan**
- 4 modes d'exécution réels : normal, subshell (**full_scan**), **fork()** C, **threads** C
- Logging simple : **echo** | **sed** » **history.log** — horodatage + 4 niveaux de sévérité
- 6 codes d'erreur avec affichage automatique de l'aide + **exit** explicite
- Helpers C dans **/tmp** — vrais **fork()** et **pthread_create()**, Thread split fonctionnel
- Fonctionnalités bonus : baseline SHA-256, auto-remédiation, JSON, CVE kernel, banner grab
- Architecture modulaire facilitant l'extension du script
- Compatibilité root/non-root avec adaptation automatique des chemins de log